

Machine Learning for IoT

Lab 3 – Training & Deployment

Exercise 1: Keyword Spotting with log-Mel Spectrogram

In Deepnote, clone the notebook *Keras API with Callbacks* to create a new notebook that trains and evaluates a model for “up/down” keyword spotting on the Mini Speech Command (MSC) dataset.

1.1 Modify the data pipeline to read only wav files of “down” and “up” keywords from the MSC train, validation, and test splits, mapping “down” to 0 and “up” to 1.

1.2 Run training using the following configurations:

Preprocessing Hyperparameters:

- Sampling rate: 16000Hz.
- STFT frame length: 40ms.
- STFT frame overlap: 50%.
- # of mel bins: 40.
- Mel lower frequency: 20Hz.
- Mel upper frequency: 4000Hz.

Training Hyperparameters:

- Batch size: 20.
- Initial learning rate: 0.01.
- End learning rate: 1e-5.
- # of epochs: 20.

Model Architecture: Sequential Convolutional Neural Network

1.2 Repeat the training & evaluation flow using different hyperparameters values:

- STFT frame length $\in [10, 50]$ ms. Try also with power-of-two values like 8ms, 16ms, 32ms.
- STFT frame step such that overlap $\in \{0\%, 25\%, 50\%, 75\%\}$.
- # of mel bins $\in [10, 40]$.
- Mel lower frequency $\in [20, 80]$ Hz.
- Mel upper frequency $\in [2000, 8000]$ Hz.

1.3 Repeat the training & evaluation flow using the ResNet8 as the model architecture (see the notebook *Keras Sub-classing API*):

1.4 For each configuration, evaluate the *SparseCategoricalAccuracy* and report the collected results in a table. Comment the collected results:

- Which is the configuration with the highest accuracy?
- How much accuracy improved after tuning the pre-processing hyperparameters?
- Which is (are) the configuration(s) that most affect(s) accuracy?

Exercise 2: Keyword Spotting with MFCCs

In Deepnote, create a modified version of the notebook of *Exercise 1* to train a model with MFCC features.

2.1 Implement a *tf.data* pipeline that computes MFCC features with the following hyperparameters:

- Sampling rate: 16000Hz.
- STFT frame length: 40ms.
- STFT frame overlap: 50%.
- # of mel bins: 40.
- Mel lower frequency: 20Hz.
- Mel upper frequency: 4000Hz.
- # of MFCCs: 10.

2.2 Run the training & evaluation flow of *Exercise 1* using the Sequential Convolutional Neural Network.

2.3 Repeat the flow using different hyperparameters values:

- STFT frame length $\in [10, 50]$ ms. Try also with power-of-two values like 8ms, 16ms, 32ms.
- STFT frame step such that overlap $\in \{0\%, 25\%, 50\%, 75\%\}$.
- # of mel bins $\in [10, 40]$.
- Mel lower frequency $\in [20, 80]$ Hz.
- Mel upper frequency $\in [2000, 8000]$ Hz.
- # of MFCCs $\in [10, 40]$.

2.4 Repeat the flow using the ResNet8 model.

2.5 For each configuration, evaluate the *SparseCategoricalAccuracy*. Report and comment the collected results.

2.6 Compare the results of *Exercises 1* and *2* and answer the following questions:

- Which is the maximum accuracy achievable?
- Which is the pre-processing technique that guarantees the maximum accuracy?
- Which is the impact of the different important optimizations (feature extraction, hyperparameters tuning, and model architecture)?

Exercise 3: TFLite Conversion

In Deepnote, create a Python notebook to convert a trained model from the TensorFlow *SavedModel* format to the *TFLite* format.

- Convert the *SavedModel* using the *TFLiteConverter*.
- Save the *TFLite* model in a new folder named *tflite_models*.
- Repeat the the TFLite conversion for the models generated in *Exercises 1* and *2*.

Exercise 4: TFLite Testing

4.1 In Deepnote, create two Python notebooks to test the *TFLite* models generated in Exercise 3:

- 1) Test Inference with log-Mel Spectrogram.
- 2) Test Inference with MFCCs.

4.2 For each test, develop an inference pipeline that:

- Read WAV files from the MSC test-set
- Apply pre-processing
- Run inference with the *TFLite* interpreter.
- Measure the categorical accuracy.
- Measure the *TFLite* model size in KB (1KB = 1024 bytes).

4.3 Deploy the tested TFLite model to the Raspberry Pi and measure the inference latency using the *inference_latency.py* script. Use the script to measure the median latency needed for pre-processing, model execution, and their sum (using batch size=1).

4.4 Summarize the collected results in the following plots:

- Accuracy (in %) vs. Latency (in ms).
- Accuracy (in %) vs. Model size (in KB).

4.5 Comment the results:

- Which is the major contribution to inference latency? Pre-processing or model execution?
- Identify the configurations belonging to the Pareto front in the Accuracy vs. Latency trade-off.
- Identify the configurations belonging to the Pareto front in the Accuracy vs. Memory trade-off.

Note: See the definition of Pareto efficiency in https://en.wikipedia.org/wiki/Pareto_efficiency.

Exercise 5: TFLite Deployment

Select one *TFLite* model obtained from *Exercise 4* and deploy it to your Raspberry Pi.

5.1 Record few examples of keywords with the USB microphone and store the audio data on disk. For recording set the # of channels to 1, the resolution to *int16*, and the sampling frequency to 48 kHz. Downsample to 16 kHz before storing the recording on disk.

5.2 In VS Code, write a Python script that reads the recorded audio data and run inference with the *TFLite* model.

5.3 Double-check the predictions of the *TFLite* model with the collected data and comment the results:

- Is the model accurate enough with real data?
- What can be done to improve the prediction quality?